

FMI Layered Standard for DAE (FMI-LS-DAE)

Contents

1. Introduction	2
1.1. Intent of this Document	2
1.2. How to read this Document	3
2. Common Concepts	3
2.1. Mathematical Notation	3
2.2. Mathematical Formulation	3
2.3. Changing between ODE and DAE Mode	4
2.3.1. The <code>enableDAEModeParameter</code> Structural Parameter	5
2.3.2. Entering DAE Mode	5
2.4. ODE with Invariants	6
3. Layered Standard Manifest File	7
3.1. Enabling DAE mode	8
3.2. Algebraic variables	8
3.3. Model Structure	8
3.4. Getting Partial Derivatives	10
3.5. Example	10
3.5.1. Robertson DAE - Semi-Explicit DAE	11
3.5.2. Robertson DAE - Explicit ODE	12
4. Appendix	14
4.1. DAE representation	14
4.1.1. Fully-Implicit DAE	14
4.1.2. Semi-Explicit Index 1 / Hessenberg index 1 DAE	14
4.1.3. Relationship between fully-implicit and semi-explicit DAE	15
4.1.4. Semi-Explicit DAE	15
4.1.5. Mass matrix DAE	15
4.1.6. Linear time-invariant DAE	15
4.1.7. Indices	15
4.1.8. Hello World DAE Example	18
4.2. Generic DAE Use Case	19
4.2.1. Modelica Example CauerLowPassAnalog	19
4.2.2. Scaling Power Grids Example	21
4.3. Invariants and Numerical Drift	22
4.3.1. Numerical Drift	22
4.3.2. Example: The Mass-Spring Oscillator	23

4.3.3. DAE Index Reduction and ODE with Invariants	24
4.3.4. Invariant and Manifold Checking	25
References	26

This layered standard on top of FMI 3.0 defines how to exchange dynamic models in differential algebraic equation form.



Although the document refers to version 3.0 of the FMI standard, everything described in this document also applies to all subsequent minor versions. For further information on compatibility, see section [Versioning and Layered Standards](#) in the FMI 3.0 specification.

Copyright © 2025-2026 The Modelica Association Project FMI.

This document is licensed under the Attribution-ShareAlike 4.0 International license. The code is released under the 2-Clause BSD License. The license text can be found in the [LICENSE.txt](#) file that accompanies this distribution.

1. Introduction

1.1. Intent of this Document

A differential-algebraic system of equations (DAE) is a system of equations that either contains differential equations and algebraic equations, or is equivalent to such a system ^[1].

- Avoid the requirement of index reduction inside of FMUs.
 - This could improve accuracy due to better drift handling.
- Avoid local nonlinear equation solvers inside of FMUs.
 - This could improve accuracy and avoid problems with different local and global error tolerances.
- Preserve the sparseness of DAE systems which is lost for the corresponding reduced ODE systems.
 - This could improve the performance by usage of the sparseness.
- Allow connections between constraint FMUs.
 - Connecting reduced ODE FMUs could lead globally to a non-solvable (singular) system but not for unreduced DAE FMUs.

1.2. How to read this Document

The standard document is in HTML allowing heavy use of in-document links: all state names, function names, many function arguments, XML elements and attributes are links to definitions or descriptions.

In key parts of this document, non-normative examples are used to help understand the standard.

Conventions used in this document:

- Non-normative text is given like this:



Especially examples are defined in this style.

- The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "MAY", and "OPTIONAL" in this document are to be interpreted as described in [RFC 2119](#) (regardless of formatting and capitalization).
- State machine states are formatted as **bold** link, e.g. [InitializationMode](#).

2. Common Concepts

This layered standard extends FMI 3.0 Model Exchange with support for semi-explicit index-1 differential-algebraic equations by introducing algebraic variables and residual equations alongside the continuous states and state derivatives from the core standard.

2.1. Mathematical Notation

This layered standard uses the same mathematical notation as provided in the FMI 3.0 standard under sub-section [Mathematical Notation](#), with the addition of the symbols introduced here.

Table 1. Additional symbols for specific variable types.

Variable	Description
$\mathbf{a}_c$	A vector of real continuous-time variables representing the continuous-time algebraic variables .

2.2. Mathematical Formulation

The DAE layered standard version **1.0.0-alpha.1** is intentionally restricted to **semi-explicit index-1 DAEs** used with FMI Model Exchange during [Continuous-Time Mode](#).

The supported formulation of DAEs is

$$\dot{\mathbf{x}}_c = \mathbf{f}_{der}(\mathbf{x}_c, \mathbf{a}_c, \mathbf{u}_c, \mathbf{p}, t)$$

$$0 = \mathbf{f}_{res}(\mathbf{x}_c, \mathbf{a}_c, \mathbf{u}_c, \mathbf{p}, t)$$

$$\mathbf{y} = \mathbf{f}_{out}(\mathbf{x}_c, \mathbf{a}_c, \mathbf{u}_c, \mathbf{p}, t)$$

where:

- $\mathbf{x}_c$ denotes the continuous-time [states](#),
- $\dot{\mathbf{x}}_c$ denotes the continuous-state [derivatives](#),
- $\mathbf{a}_c$ denotes the [algebraic variables](#),
- $\mathbf{u}_c$ denotes the continuous-time [inputs](#),
- $\mathbf{p}$ denotes the [parameters](#),
- t denotes time,
- $\mathbf{f}_{res}$ denotes the algebraic equations expressed as [residuals](#), and
- $\mathbf{y}$ denotes the [outputs](#).

The equations $\dot{\mathbf{x}}_c = \mathbf{f}_{der}(\mathbf{x}_c, \mathbf{a}_c, \mathbf{u}_c, \mathbf{p}, t)$ are the differential equations of the model, which can be acquired with the usual way of either

- calling `fmi3Get` on the derivative variables provided in the extended `<ModelStructure>` defined in [Section 3](#), or
- calling `fmi3GetContinuousStateDerivatives`.

The equations $0 = \mathbf{f}_{res}(\mathbf{x}_c, \mathbf{a}_c, \mathbf{u}_c, \mathbf{p}, t)$ are represented in the FMU by [residual](#) variables and constrain the algebraic variables together with the other continuous-time variables of the model. These can be acquired by calling `fmi3Get` on the [residual](#) variables provided in the extended `<ModelStructure>` defined in [Section 3](#).

For the DAE to be index 1 in this layered standard means that the algebraic equations must be solvable for the algebraic variables. In particular, the Jacobian of the algebraic equations with respect to the algebraic variables must be non-singular.

Consequently, for given values of $\mathbf{x}_c$, $\mathbf{u}_c$, $\mathbf{p}$, and t , the algebraic equations determine $\mathbf{a}_c$. The importer is responsible for determining values of the algebraic variables that satisfy

$$\mathbf{f}_{res}(\mathbf{x}_c, \mathbf{a}_c, \mathbf{u}_c, t, \mathbf{p}) = 0$$

while the FMU provides the continuous-state derivatives and outputs according to the equations above.

This version of the layered standard does not define support for more general DAE formulations, including fully implicit DAEs, higher-index DAEs, or formulations that require differentiation of algebraic constraints. Such formulations must first be transformed into the supported semi-explicit index-1 form before they can be represented by this version of the layered standard.

The DAE formulation defined by this layered standard applies only while the Model Exchange FMU is in [Continuous-Time Mode](#). Behavior associated with discontinuities, event handling, and DAE formulations outside Continuous Time Mode is outside the scope of version `1.0.0-alpha.1`.

2.3. Changing between ODE and DAE Mode

An FMU that implements this layered standard is simultaneously a valid ODE FMU and a DAE FMU.

To remain backward-compatible with importers that do not support this layered standard, the FMU **defaults to ODE mode** on instantiation. An importer that is aware of this layered standard must explicitly switch the FMU into DAE mode before use.

2.3.1. The `enableDAEModeParameter` Structural Parameter

Switching between ODE and DAE behavior is controlled by a [structural parameter](#) called `enableDAEModeParameter`.

A structural parameter (`causality = structuralParameter`) may only be set during [Configuration Mode](#) or [Reconfiguration Mode](#). This constraint ensures that the importer can re-examine the model structure from the manifest and reinitialize its solver before any computation takes place.

The variable `enableDAEModeParameter` is declared in the `modelDescription.xml` as a `Boolean` under `<ModelVariables>`. The `name` and `valueReference` is assigned freely by the FMU exporter; the importer locates the parameter by its `valueReference` declared in the `<EnableDAEParameter>` element of `fmi-ls-manifest.xml`. The `description` attribute is optional.

An example of how the `enableDAEModeParameter` is declared:

```
<Boolean name="enableDAE"
  valueReference="123"
  causality="structuralParameter"
  variability="tunable"
  start="false"
  description="Set to true to enable DAE mode as defined by FMI-LS-DAE."/>
```

Table 2. `enableDAEModeParameter` structural parameter values.

Value	Meaning
<code>false</code> (default)	The FMU behaves as a standard ODE Model Exchange FMU. All DAE-specific information in the layered standard manifest is ignored by the importer.
<code>true</code>	The FMU behaves as a DAE FMU. The importer must read the <code><AlgebraicVariables></code> , <code><ModelStructure></code> , and <code><Residual></code> elements from the layered standard manifest and use them to drive the simulation.

2.3.2. Entering DAE Mode

Because `enableDAEModeParameter` is a structural parameter, the importer must change it inside a configuration or reconfiguration scope.

2.3.2.1. At Instantiation (Configuration Mode)

The typical flow for an importer that uses the FMU in DAE mode from the start is:

1. Call `fmi3InstantiateModelExchange(...)` to create the FMU instance.
2. Call `fmi3EnterConfigurationMode(instance)` to enter [Configuration Mode](#).
3. Call `fmi3SetBoolean(instance, {vr}, 1, {true}, 1)` to enable DAE mode, where `vr` is the

`valueReference` of `enableDAEModeParameter`.

4. Call `fmi3ExitConfigurationMode(instance)` to return to the `Instantiated` state.
5. Continue with the normal FMI initialization sequence (`fmi3EnterInitializationMode`, etc.).

It should be emphasized that initialization in DAE mode is not yet fully specified. This means that when the FMU is initialized in DAE mode, either

- the FMU will lift the `residual` equations to be solved by the importer, or
- the FMU will still initialize in ODE mode, or
- the FMU will error out.

2.3.2.2. During Simulation (Reconfiguration Mode)

When `variability="tunable"`, an importer may also switch between ODE and DAE mode at runtime by entering `Reconfiguration Mode` from `Event Mode`:

1. Call `fmi3EnterConfigurationMode(instance)` from Event Mode to enter Reconfiguration Mode.
2. Call `fmi3SetBoolean(instance, {vr}, 1, {newValue}, 1)` with the new mode value.
3. Call `fmi3ExitConfigurationMode(instance)` to return to Event Mode.



After switching modes, the importer is responsible for reinitializing its solver, and ensuring a consistent state before resuming the simulation.

2.4. ODE with Invariants

Many models possess quantities that remain constant along the exact solution, such as the total energy of a closed mechanical system, the total mass in a chemical reaction, or the distance between two bodies connected by a rigid rod. Such quantities are called **invariants** of the model.

Invariants also arise from index reduction. Many physical systems, for example in multibody dynamics, are naturally modeled as high-index DAEs. Reducing the index removes algebraic constraints from the equations that are actually solved, and those constraints become invariants of the reduced system.

Numerical integration does not preserve invariants exactly. Even when the initial values satisfy an invariant, the accumulated error of the integration makes the computed solution **drift** away from the manifold defined by that invariant. Exposing the invariants allows an importer to monitor this drift and to correct for it. See [Section 4.3](#) for a detailed discussion of numerical drift and of the correction methods an importer can apply.

A model with invariants is described by the equations of [Section 2.2](#), together with

$$\mathbf{c} = \mathbf{f}_{inv}(\mathbf{x}_c, \mathbf{a}_c, \mathbf{u}_c, \mathbf{p}, t)$$

where $\mathbf{f}_{inv}$ denotes the invariants of the model and $\mathbf{c}$ is constant along the exact solution.

The invariants of an ODE or of a semi-explicit DAE are exposed as variables of the

`modelDescription.xml` and are listed in the `<Invariant>` element of the extended `<ModelStructure>`, see [Section 3](#). An invariant can be scalar or vectorial. Invariants are independent of the [residuals](#), and the same variable may be referenced by both an `<Invariant>` and a `<Residual>` element.

The FMU can optionally also expose the correction $\delta = \tilde{\mathbf{x}}_c - \mathbf{x}_c$ between the integrated state $\mathbf{x}_c$ and the state $\tilde{\mathbf{x}}_c$ projected back onto the invariant manifold.

3. Layered Standard Manifest File

This layered standard requires the use of a layered standard manifest file and it shall be stored inside the FMU at the following path: `/extra/org.fmi-standard.fmi-ls-dae/fmi-ls-manifest.xml`.

Several new elements are included as direct children of the root element in the layered standard manifest file to describe the DAE formulation.

Table 3. `fmiDAEManifest` child elements.

Element	Description
<code><EnableDAEParameter></code>	Required element referencing the structural parameter in <code>modelDescription.xml</code> that enables DAE mode.
<code><AlgebraicVariables></code>	List of all algebraic variables exposed for the DAE formulation.
<code><ModelStructure></code>	Defines the structure of the DAE formulation for the model. Especially, the ordered lists of outputs , continuous-time states , and the initial unknowns (the unknowns during InitializationMode) are re-defined and overwrite the corresponding ModelStructure present in the <code>modelDescription.xml</code> , with the addition of the residuals element. The algebraic variables can be included as dependencies, and can also be initial unknowns.
[Annotations]	Optional annotations for the top-level element.

The XML attributes of `<fmiDAEManifest>` are:

Table 4. `<fmiDAEManifest>` attribute details.

Attribute	Namespace	Value	Description
<code>fmi-ls-name</code>	http://fmi-standard.org/fmi-ls-manifest	<code>org.fmi-standard.fmi-ls-dae</code>	Name of the layered standard in reverse domain name notation.
<code>fmi-ls-version</code>	http://fmi-standard.org/fmi-ls-manifest	<code>1.0.0-alpha.1</code>	Version of the layered standard. This layered standard uses semantic versioning, as defined in [PW13] .
<code>fmi-ls-description</code>	http://fmi-standard.org/fmi-ls-manifest	<code>Layered standard for DAE support in FMI.</code>	String with a brief description of the layered standard that is suitable for display to users.

3.1. Enabling DAE mode

The element `<EnableDAEParameter>` references the [structural parameter](#) in the `modelDescription.xml` that enables DAE mode. See [Section 2.3.1](#) for the full semantics and the calling sequences used to change the mode.

Table 5. Attributes to `<EnableDAEParameter>` element.

Attribute	Description
<code>valueReference</code>	The value reference of the structural parameter present in the <code><ModelVariables></code> of the <code>modelDescription.xml</code> that enables DAE mode.

3.2. Algebraic variables

The element `<AlgebraicVariables>` defines the list of the algebraic variables.

Table 6. `AlgebraicVariables` elements.

Element	Description
<code>AlgebraicVariable</code>	An <code><AlgebraicVariable></code> present in the <code><ModelVariables></code> element of the <code>modelDescription.xml</code> .

Each `<AlgebraicVariable>` has only one attribute defining the value reference.

Table 7. Attributes to `<AlgebraicVariable>` element.

Attribute	Description
<code>valueReference</code>	The value reference for the algebraic variable present in the <code><ModelVariables></code> of the <code>modelDescription.xml</code> .



In the core Model Exchange standard there exists the API function `fmi3GetNominalsOfContinuousStates`, which only holds for the continuous time states. There is currently no extension of the API for `fmi-ls-dae`. The algebraic variable could still have a nominal value defined by the `nominal` attribute for variables under `<ModelVariables>` of the `modelDescription.xml`, but in this case the nominal is constant.

3.3. Model Structure

The structure of the model for the DAE-formulation is defined in element `<ModelStructure>`. It defines the [dependencies](#) between variables. The `<ModelStructure>` element is extended with two additional elements - `<Residual>` and `<Invariant>` - and the optional dependencies can now include algebraic variables.

An FMU that follows this layered standard must expose all residuals using the `<Residual>` element. Invariants of the model can optionally be exposed using the `<Invariant>` element.

The `<ModelStructure>` defined here extends the one from the core FMI standard. All elements

defined in the core standard (`<Output>`, `<ContinuousStateDerivative>`, `<ClockedState>`, `<InitialUnknown>`, `<EventIndicator>`) retain their existing semantics in this manifest, with the additions that:

- Each element's `dependencies` attribute may now also reference [algebraic variables](#).
- The `<InitialUnknown>` element is expanded so that [algebraic variables](#) can be included as unknowns.

The `<ModelStructure>` element is extended with two additional elements, `<Residual>` and `<Invariant>`, described below.

Table 8. Additional ModelStructure elements.

Element	Description
<code>Residual</code>	Ordered list of residual equations (constraints). Each residual element must reference a unique variable present in the <code><ModelVariables></code> of the <code>modelDescription.xml</code> . It is the importer's responsibility to find the values of the algebraic variables such that the residuals are 0. The functional dependency is defined as: $0 := \mathbf{f}_{res}(\mathbf{x}_c, \mathbf{a}_c, \mathbf{u}_c, \mathbf{p}, t)$
<code>Invariant</code>	Ordered list of invariants of the model, see Section 2.4 . Each <code><Invariant></code> element references the variable that computes an invariant of the FMU, which must be present in the <code><ModelVariables></code> of the <code>modelDescription.xml</code> . The invariant can be scalar or vectorial. Invariants are independent of the residuals , and the same variable may also be referenced by a <code><Residual></code> element. The dependencies on continuous states, inputs, algebraic variables, and parameters can optionally be included [MN2018] . The functional dependency is defined as: $\mathbf{const} := \mathbf{f}_{inv}(\mathbf{x}_c, \mathbf{a}_c, \mathbf{u}_c, \mathbf{p}, t)$

Table 9. Attributes to `<Residual>` element.

Attribute	Description
<code>valueReference</code>	The value reference of the residual variable $v_{unknown}$, which must be present in the <code><ModelVariables></code> element of the <code>modelDescription.xml</code> .

Attribute	Description
<code>dependencies</code>	Optional attribute defining the algebraic dependencies as a list of value references of the knowns $\mathbf{v}_{known}$ that this residual equation directly depends on. Knowns $\mathbf{v}_{known}$ in Continuous-Time Mode (ME) for <code><Residual></code> elements are: • inputs (variables with <code>causality = input</code>), • continuous states, • parameters (variables with <code>causality = parameter</code>), • algebraic variables (variables listed in an <code><AlgebraicVariables></code> element), • <code>independent</code> variable (usually time; <code>causality = independent</code>). If <code>dependencies</code> is not present, it must be assumed that the residual depends on all knowns. If <code>dependencies</code> is present as an empty list, the residual depends on none of the knowns. Otherwise the residual depends on the knowns defined by the given value references.
<code>dependenciesKind</code>	See the description of <code>dependenciesKind</code> in the core FMI standard.

3.4. Getting Partial Derivatives

The partial derivative API functions `fmi3GetDirectionalDerivative` and `fmi3GetAdjointDerivative` are used as defined in the core FMI standard, with the following extensions for DAE FMUs:

- `Residual` variables can appear as unknowns.
- `Algebraic variables` can appear as knowns for any unknown.

3.5. Example

As an example, consider a problem from chemical kinetics due to Robertson [\[Robertson1966\]](#), which is also included among the IDA example programs [\[Hindmarsh2026IDAExamples\]](#), and implemented as the [Reference FMU Roberts](#). The DAE is given by

$$\begin{aligned}\frac{dy_1}{dt} &= -0.04y_1 + 10^4 y_2 y_3 \\ \frac{dy_2}{dt} &= 0.04y_1 - 10^4 y_2 y_3 - 3 \times 10^7 y_2^2 \\ 0 &= y_1 + y_2 + y_3 - 1\end{aligned}$$

with initial conditions

$$y_1(0) = 1, \quad y_2(0) = y_3(0) = 0.$$

The variable y_3 does not appear differentiated in any equation and is therefore an algebraic variable. In the notation of [Section 2.2](#), the continuous-time states and algebraic variables are

$$\mathbf{x}_c = \begin{pmatrix} y_1 y_2 \end{pmatrix}, \quad \mathbf{a}_c = \begin{pmatrix} y_3 \end{pmatrix}.$$

There are no inputs or parameters in this example, i.e., $\mathbf{u}_c$ and $\mathbf{p}$ are empty. The equations also have no explicit dependence on time.

3.5.1. Robertson DAE - Semi-Explicit DAE

Using the semi-explicit formulation defined in [Section 2.2](#), the differential equations are

$$\dot{\mathbf{x}}_c = \mathbf{f}_{der}(\mathbf{x}_c, \mathbf{a}_c, \mathbf{u}_c, \mathbf{p}, t)$$

with

$$\mathbf{f}_{der}(\mathbf{x}_c, \mathbf{a}_c, \mathbf{u}_c, \mathbf{p}, t) = \begin{pmatrix} -0.04 y_1 + 10^4 y_2 y_3 \\ 0.04 y_1 - 10^4 y_2 y_3 - 3 \times 10^7 y_2^2 \end{pmatrix}$$

and the algebraic equation is

$$0 = \mathbf{f}_{res}(\mathbf{x}_c, \mathbf{a}_c, \mathbf{u}_c, \mathbf{p}, t)$$

with

$$r := \mathbf{f}_{res}(\mathbf{x}_c, \mathbf{a}_c, \mathbf{u}_c, \mathbf{p}, t) = y_1 + y_2 + y_3 - 1$$

The initial values in this notation are

$$\mathbf{x}_c(0) = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad \mathbf{a}_c(0) = \begin{pmatrix} 0 \end{pmatrix}$$

The Jacobian of the residual equation with respect to the algebraic variable is

$$\frac{\partial \mathbf{f}_{res}}{\partial \mathbf{a}_c} = \begin{pmatrix} 1 \end{pmatrix}$$

which is non-singular. Hence, the algebraic equation can be locally solved for $\mathbf{a}_c$, and the Robertson problem is an index-1 DAE in the representation supported by this layered standard.

Assume the FMU has the following variables:

Variable	valueReference
t	0
dae	1
y_1	2
$\dot{y}_1$	3
y_2	4
$\dot{y}_2$	5

Variable	valueReference
y_3	6
r	7

Here, y_1 and y_2 are the components of $\mathbf{x}_c$, while y_3 is the single component of $\mathbf{a}_c$. The residual $\mathbf{f}_{res}$ is represented by the variable r .

The DAE-specific manifest elements are:

```
<EnableDAEParameter valueReference="1"/>

<AlgebraicVariables>
  <AlgebraicVariable valueReference="6"/>
</AlgebraicVariables>

<ModelStructure>
  <ContinuousStateDerivative
    valueReference="3"
    dependencies="2 4 6"
    dependenciesKind="constant constant constant"/>
  <ContinuousStateDerivative
    valueReference="5"
    dependencies="2 4 6"
    dependenciesKind="constant constant constant"/>
  <InitialUnknown
    valueReference="3"
    dependencies="2 4 6"
    dependenciesKind="constant constant constant"/>
  <InitialUnknown
    valueReference="5"
    dependencies="2 4 6"
    dependenciesKind="constant constant constant"/>
  <InitialUnknown
    valueReference="6"
    dependencies="2 4 6"
    dependenciesKind="constant constant constant"/>
  <Residual
    valueReference="7"
    dependencies="2 4 6"
    dependenciesKind="constant constant constant"/>
</ModelStructure>
```

3.5.2. Robertson DAE - Explicit ODE

Because the residual equation is locally solvable for $\mathbf{a}_c$, the algebraic variable can be eliminated to obtain an explicit ODE.

For fixed $\mathbf{x}_c$, $\mathbf{u}_c$, $\mathbf{p}$, and t , consider the mapping

$$\mathbf{a}_c \mapsto \mathbf{f}_{res}(\mathbf{x}_c, \mathbf{a}_c, \mathbf{u}_c, \mathbf{p}, t)$$

Its inverse with respect to $\mathbf{a}_c$, evaluated at residual zero, gives

$$\mathbf{a}_c = (\mathbf{f}_{res}(\mathbf{x}_c, \cdot, \mathbf{u}_c, \mathbf{p}, t))^{-1}(0)$$

For the Robertson problem,

$$(\mathbf{f}_{res}(\mathbf{x}_c, \cdot, \mathbf{u}_c, \mathbf{p}, t))^{-1}(0) = 1 - y_1 - y_2$$

and therefore

$$y_3 = 1 - y_1 - y_2$$

Substituting this inverse into the differential equations gives the explicit ODE

$$\dot{\mathbf{x}}_c = \mathbf{f}_{der}(\mathbf{x}_c, (\mathbf{f}_{res}(\mathbf{x}_c, \cdot, \mathbf{u}_c, \mathbf{p}, t))^{-1}(0), \mathbf{u}_c, \mathbf{p}, t)$$

For the Robertson problem, this becomes

$$\dot{\mathbf{x}}_c = \begin{pmatrix} -0.04 y_1 + 10^4 y_2 (1 - y_1 - y_2) \\ 0.04 y_1 - 10^4 y_2 (1 - y_1 - y_2) - 3 \times 10^7 y_2^2 \end{pmatrix}$$

The quantity corresponding to the algebraic variable in the DAE formulation can still be computed in the ODE representation as

$$y_3 = (\mathbf{f}_{res}(\mathbf{x}_c, \cdot, \mathbf{u}_c, \mathbf{p}, t))^{-1}(0) = 1 - y_1 - y_2$$

The model is then represented as an ODE Model Exchange FMU. Using the value references of the Robertson FMU, the relevant variables are:

Variable	valueReference
t	0
y_1	2
$\dot{y}_1$	3
y_2	4
$\dot{y}_2$	5
y_3	6

In the ODE representation, y_3 is computed from y_1 and y_2 and is not listed as an `<AlgebraicVariable>`.

The corresponding regular ODE `ModelStructure` available in the `modelDescription.xml` is:

```
<ModelStructure>
  <ContinuousStateDerivative
    valueReference="3"
    dependencies="2 4"
    dependenciesKind="dependent dependent"/>
```

```

<ContinuousStateDerivative
  valueReference="5"
  dependencies="2 4"
  dependenciesKind="dependent dependent"/>
<InitialUnknown
  valueReference="3"
  dependencies="2 4"
  dependenciesKind="dependent dependent"/>
<InitialUnknown
  valueReference="5"
  dependencies="2 4"
  dependenciesKind="dependent dependent"/>
</ModelStructure>

```

4. Appendix

This appendix contains some general notes and discussions on the topic of DAEs.

4.1. DAE representation

There are different forms of DAEs.

For simplicity discontinuous states are currently disregarded.

4.1.1. Fully-Implicit DAE

The generic *fully-implicit DAE*, or *implicit DAE*, formulation:

$$F(\dot{z}(t), z(t), t) = 0$$

where $z(t)$ represents state and additional variables and t is time.

A similar and sometimes more useful representation:

$$F(\dot{x}(t), x(t), y(t), t) = 0$$

with dynamic variables $x(t)$, algebraic variables $y(t)$ and time t .

Note that $x(t)$ is not necessarily the state variable, more on this in [Section 4.1.7.2](#).

4.1.2. Semi-Explicit Index 1 / Hessenberg index 1 DAE

The *semi-explicit index 1*, also called *Hessenberg index 1*, DAE representation

$$\begin{aligned}\dot{x}(t) &= f(x(t), y(t), t) \\ 0 &= g(x(t), y(t), t)\end{aligned}$$

with g solvable for y , $\det\left(\frac{\partial}{\partial y} g\right) \neq 0$.

Here f is called the *differential equations* and g the *algebraic equations*.

4.1.3. Relationship between fully-implicit and semi-explicit DAE

A fully implicit DAE

$$F(\dot{x}(t), x(t), t) = 0$$

can always be rewritten as a semi-explicit DAE by introducing a new algebraic variable x'

$$\begin{aligned}\dot{x}(t) &= x'(t) \\ 0 &= F(x'(t), x(t), t)\end{aligned}$$

4.1.4. Semi-Explicit DAE

The *semi-explicit DAE* representation

$$\begin{aligned}\dot{x}(t) &= f(x(t), y(t), t) \\ 0 &= g^1(x(t), y(t), t) \\ 0 &= g^2(x(t), y(t), t) \\ 0 &= g^3(x(t), y(t), t) \\ &\vdots\end{aligned}$$

with g^1 solvable for y , $\det\left(\frac{\partial}{\partial y} g^1\right) \neq 0$ and $\left\{\frac{d}{dt} g_i^k\right\} \subseteq \{g_i^{k-1}\}$, $k > 1$.

4.1.5. Mass matrix DAE

$$M\dot{x}(t) = f(x(t), t)$$

where *mass matrix* M is singular.

An equivalent explicit formulation:

$$\begin{bmatrix} M_x & 0 \\ 0 & 0 \end{bmatrix} \cdot \begin{bmatrix} \dot{x} \\ 0 \end{bmatrix} = \begin{bmatrix} \hat{f}(x(t), y(t), t) \\ g(x(t), y(t), t) \end{bmatrix}$$

See [\[cui2020mass\]](#).

4.1.6. Linear time-invariant DAE

$$E\dot{x}(t) = Ax(t)$$

where matrix E is singular.

4.1.7. Indices

There are multiple, sometimes closely related, definitions of indices for DAE systems. Usually these indices are a measure of how far from an ODE the DAE is.

4.1.7.1. Differential Index

The minimum number of times a DAE

$$F(\dot{x}(t), x(t), t) = 0$$

has to be differentiated with respect to time t to be able to determine $\dot{x}(t)$ as a function of t and x is called the *differential index*.

See [\[hairer2006numerical\]](#).

4.1.7.2. Perturbation Index

The *perturbation index* measures the sensitivity of the solution of a DAE to perturbations of its right-hand side. The DAE

$$F(\dot{x}(t), x(t), t) = 0$$

has perturbation index p along a solution $x(t)$ if p is the smallest non-negative integer such that the solution $\tilde{x}(t)$ of the perturbed system

$$F(\dot{\tilde{x}}(t), \tilde{x}(t), t) = \delta(t)$$

satisfies

$$\|x(t) - \tilde{x}(t)\| \leq C \left(\|x(t_0) - \tilde{x}(t_0)\| + \max_{t_0 \leq \xi \leq t} \|\delta(\xi)\| + \max_{t_0 \leq \xi \leq t} \|\dot{\delta}(\xi)\| + \dots + \max_{t_0 \leq \xi \leq t} \|\delta^{(p-1)}(\xi)\| \right)$$

for some constant C and a sufficiently small and smooth perturbation $\delta(t)$. A DAE with perturbation index greater than 1 is called a *higher-index DAE*.

See [\[hairer2006numerical\]](#).

4.1.7.2.1. Example of Structural Singularity

[\[cellier2006continuous, chapter 7.7 Structural Singularity Elimination\]](#) provides a simple electric circuit model that is a higher-index DAE.

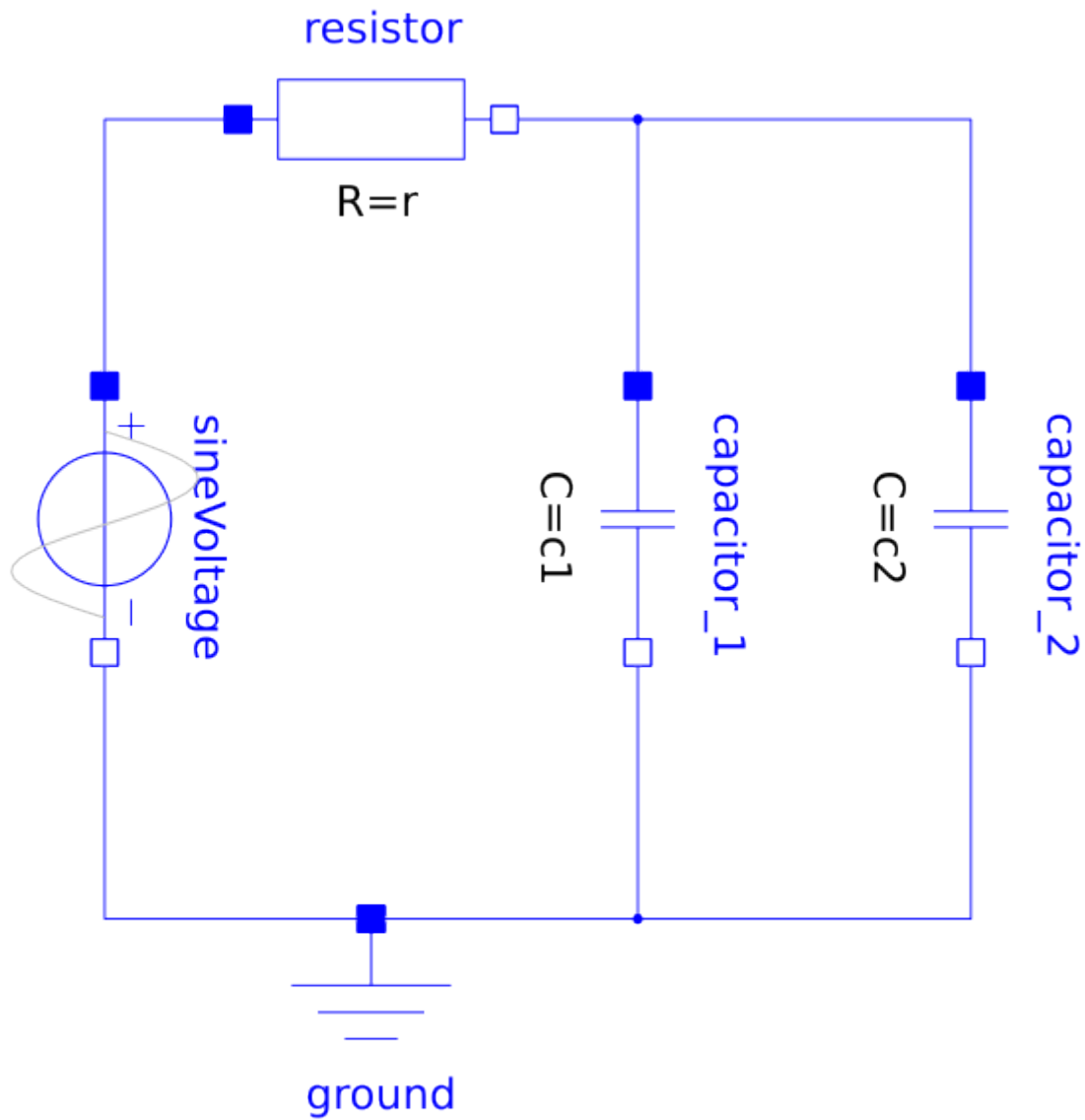


Figure 1. Modelica model of an electric circuit with two capacitors in parallel.

The circuit has two capacitors in parallel and can be represented with equations:

$$\begin{aligned}
 v_0 &= f(t) \\
 v_R &= R \cdot i_0 \\
 i_1 &= C_1 \cdot \dot{v}_1 \\
 i_2 &= C_2 \cdot \dot{v}_2 \\
 v_0 &= v_R + v_1 \\
 v_2 &= v_1 \\
 i_0 &= i_1 + i_2
 \end{aligned}$$

When both capacitive voltages v_1 and v_2 are chosen as state variables, equation $v_2 = v_1$ has no unknowns left, so it is a *constraint equation*.

There are different ways to solve this. The modeler could change the causality or exporting tools can try to reduce the perturbation index symbolically ^[2] to end up with a consistent initialization for the system.

4.1.8. Hello World DAE Example

A simple initial-value problem

$$\begin{aligned}\dot{x} &= y \\ y + 2z &= x \\ y - 3z &= 2x\end{aligned}$$

with variables $x, y, z \in \mathbb{R}$ and start condition

$$\begin{pmatrix} x(t_0) \\ y(t_0) \\ z(t_0) \end{pmatrix} = \begin{pmatrix} x_0 \\ y_0 \\ z_0 \end{pmatrix} \text{ and } \dot{x}(t_0) = \dot{x}_0$$

at time t_0 . For this specific example a start condition x_0 is sufficient to calculate consistent start values. The independent variable (usually time) t is omitted for simplicity.

4.1.8.1. Hello World DAE - Fully-Implicit DAE

The fully-implicit representation

$$F(\dot{x}, x, t) = 0$$

is given by

$$F : \mathbb{R}^3 \times \mathbb{R}^3 \times \mathbb{R} \rightarrow \mathbb{R}^3, \begin{pmatrix} x \\ \dot{x} \\ t \end{pmatrix} \mapsto \begin{pmatrix} \dot{x}_1 - x_2 \\ x_2 + 2x_3 - x_1 \\ x_2 - 3x_3 - 2x_1 \end{pmatrix}$$

4.1.8.2. Hello World DAE - Semi-Explicit DAE

The semi-explicit representation

$$\begin{aligned}\dot{x} &= f(x, y, t) \\ 0 &= g(x, y, t)\end{aligned}$$

is given by

$$\begin{aligned}f : \mathbb{R} \times \mathbb{R}^2 \times \mathbb{R} \rightarrow \mathbb{R}, (x, y, t) &\mapsto \begin{pmatrix} y_1 \end{pmatrix} \\ g : \mathbb{R} \times \mathbb{R}^2 \times \mathbb{R} \rightarrow \mathbb{R}, (x, y, t) &\mapsto \begin{pmatrix} y_1 + 2y_2 - x \\ y_1 - 3y_2 - 2x \end{pmatrix}\end{aligned}$$

4.1.8.3. Hello World DAE - Explicit ODE

This simple system could also be transformed into an explicit ODE formulation

$$\dot{x} = \frac{7}{5}x$$

with local variables y and z

$$\begin{aligned}y &= \frac{7}{5}x \\z &= -\frac{x}{5}\end{aligned}$$

and exported as an ODE ModelExchange FMU.

4.2. Generic DAE Use Case

There are a number of possible reasons to export a model as a system of differential-algebraic equations (DAE) instead of a system of ordinary differential equations (ODE):

- The exporting tool doesn't support methods to transform a DAE into ODE representation.
- The time needed to perform symbolic transformation to ODE representation takes a significant amount of time for large systems.
- Large system models are usually characterized by a high degree of sparsity. Some DAE solvers can utilize sparsity to reduce simulation time.
- More efficient simulation of models with large algebraic loops for DAE representation.

See [\[braun2017solving\]](#).

4.2.1. Modelica Example CauerLowPassAnalog

The Modelica model [Modelica.Electrical.Analog.Examples.CauerLowPassAnalog](#) from the Modelica Standard Library ^[3]

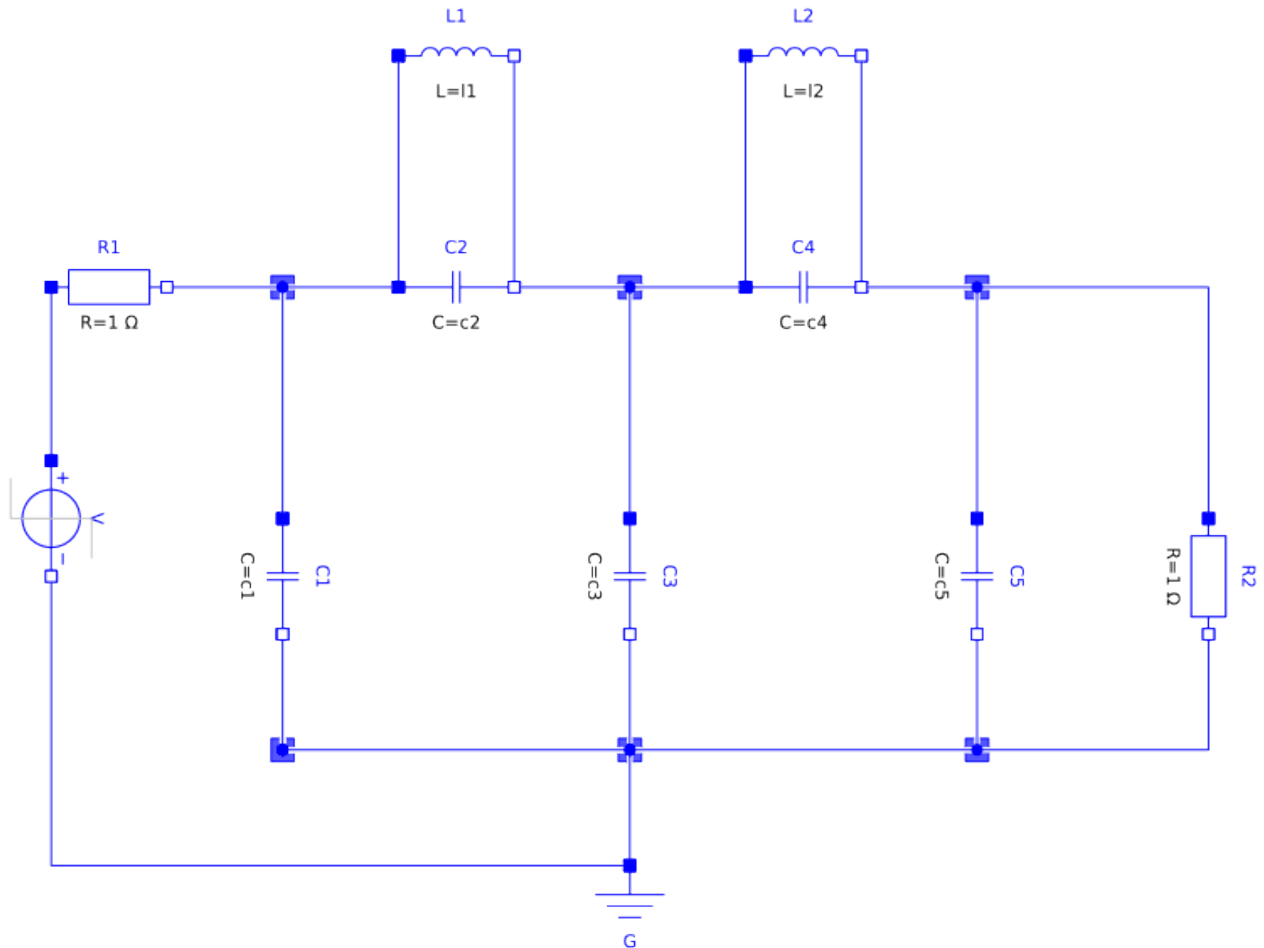


Figure 2. Graphical representation of the Cauer low-pass-filter
Modelica.Electrical.Analog.Examples.CauerLowPassAnalog.

has structural singularities.

To produce a complete and consistent set of initial conditions for the DAE, exporting tools usually use a symbolic index reduction algorithm.

The model has nine state candidates

```

1: L1.v (unit = "V" stateSelect=StateSelect.never ) "Voltage drop of the two pins (=
p.v - n.v)" type: Real
2: L2.v (unit = "V" stateSelect=StateSelect.never ) "Voltage drop of the two pins (=
p.v - n.v)" type: Real
3: C2.v (start = 0.0 unit = "V" ) "Voltage drop of the two pins (= p.v - n.v)" type:
Real
4: C4.v (start = 0.0 unit = "V" ) "Voltage drop of the two pins (= p.v - n.v)" type:
Real
5: L1.i (start = 0.0 unit = "A" fixed = true ) "Current flowing from pin p to pin n"
type: Real
6: L2.i (start = 0.0 unit = "A" fixed = true ) "Current flowing from pin p to pin n"
type: Real
7: C1.v (start = 0.0 unit = "V" fixed = true ) "Voltage drop of the two pins (= p.v -
n.v)" type: Real

```

```
8: C5.v (start = 0.0 unit = "V" fixed = true ) "Voltage drop of the two pins (= p.v - n.v)" type: Real
9: C3.v (start = 0.0 unit = "V" fixed = true ) "Voltage drop of the two pins (= p.v - n.v)" type: Real
```

and four constraint equations

```
1: L2.v = C3.v - C5.v
2: L1.v = C1.v - C3.v
3: C4.v = L2.v
4: C2.v = L1.v
```

Modelica tools that want to export an ODE ModelExchange FMU need to reduce the perturbation index to index 1 to end up with an ODE representation of the DAE system.

One option to achieve this is the so called dummy derivative method. From the set of state candidates, five state variables are chosen (e.g. `C1.v`, `C3.v`, `C5.v`, `L1.i`, `L2.i`) and the constraint equations

$$\begin{aligned}C2.v &= L1.v \\L1.v &= C1.v - C3.v \\C4.v &= L2.v \\L2.v &= C3.v - C5.v\end{aligned}$$

are differentiated

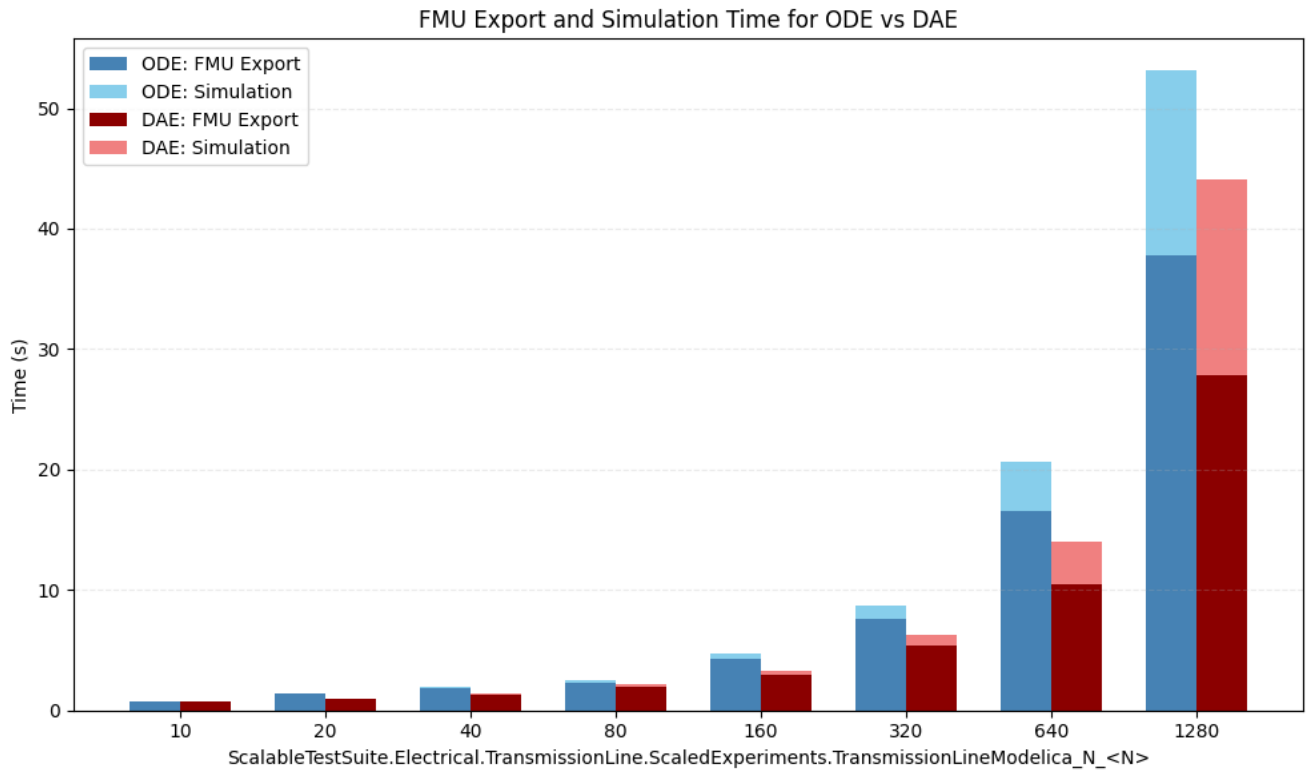
$$\begin{aligned}\frac{d}{dt}C2.v &= \frac{d}{dt}L1.v \\ \frac{d}{dt}L1.v &= \frac{d}{dt}C1.v - \frac{d}{dt}C3.v \\ \frac{d}{dt}C4.v &= \frac{d}{dt}L2.v \\ \frac{d}{dt}L2.v &= \frac{d}{dt}C3.v - \frac{d}{dt}C5.v\end{aligned}$$

which results in four additional dummy states (`$DER.L1.v`, `$DER.L2.v`, `$DER.C2.v`, `$DER.C4.v`). Here the Modelica keyword `stateSelect` influences how variables are picked.

4.2.2. Scaling Power Grids Example

For simulation models representing power grids on a national or larger scale, traditional ODE solvers could struggle with convergence and performance. Also, the time needed to perform symbolic transformation from DAE to ODE is growing with the number of equations.

See [for example](#) [Modelica models ScalableTestSuite.Electrical.TransmissionLine.ScaledExperiments.TransmissionLineModelica_N_10](#) to [ScalableTestSuite.Electrical.TransmissionLine.ScaledExperiments.TransmissionLineModelica_N_1280](#) from ScalableTestSuite ^[4].



The simulation was performed using `OpenModelica v1.27-dev-92` with the following settings:

- DAE export: `--daeMode`
- Method: Sundials IDA ^[5]

Since OpenModelica does not support FMI 3.0 or FMI-LS-DAE at the time of writing, the export and simulation times were measured using OpenModelica’s default C export and simulation.

4.3. Invariants and Numerical Drift

This section gives the background for [Section 2.4](#).

4.3.1. Numerical Drift

When a mathematical model is derived for a physical system, the fundamental conservation laws (such as energy or momentum) or geometric properties are directly embedded into the differential equations. These conservation laws define the system’s **invariants** (quantities that remain constant over time). Examples include total energy in a closed system, total mass in a chemical reaction, or angular momentum in a gravitational field.

The analytical solution to the differential equation respects these invariants perfectly. However, when numerical solvers (like Euler’s method, Runge-Kutta, or BDF methods) are used to simulate the differential equations, *numerical drift* is introduced.

Numerical solvers solve the ODEs/DAEs by approximating the continuous-time variable with a series of discrete steps. At each step, they calculate an update based on the current state, inevitably accumulating a small, local truncation error. While high-order methods can make this error very small for the **state variables** themselves, they are generally **not designed to preserve arbitrary**

invariants of the system.

This means that even if the initial condition perfectly satisfies an invariant (e.g., total energy = E_0), after hundreds or thousands of time steps, the computed solution may slowly but surely "drift" away from the manifold defined by that invariant. The solver is effectively solving a *nearby* differential equation, one whose true solution does not possess the same conserved quantities.

As a consequence of numerical drift, the simulation can produce results that violate fundamental physical laws. A satellite's orbit might slowly decay or escape when it should be stable. In molecular dynamics, the temperature (related to kinetic energy) might artificially rise or fall, rendering the simulation useless. Also, small drifts can accumulate, leading to a complete breakdown of the solution over long time horizons. The numerical solution may blow up or behave chaotically even when the true solution is well-behaved.

4.3.2. Example: The Mass-Spring Oscillator

Consider a physical system consisting of a mass m attached to a spring with a spring constant k . When the mass is displaced by a distance x from its equilibrium position, the spring exerts a restoring force given by Hooke's law:

$$F = -kx$$

Applying Newton's second law ($m\ddot{x} = F$) gives:

$$m\ddot{x} = -kx \quad \Rightarrow \quad \ddot{x} + \frac{k}{m}x = 0$$

To solve this numerically, we convert it into a system of two first-order ODEs by introducing velocity ($v = \dot{x}$):

$$\begin{aligned}\dot{x} &= v, \\ \dot{v} &= -\frac{k}{m}x\end{aligned}$$

The total mechanical energy $I(x, v)$ (the Hamiltonian) of this closed system is the sum of kinetic and potential energy, which must remain strictly constant:

$$I(x, v) = \frac{1}{2}mv^2 + \frac{1}{2}kx^2 = \text{constant}$$

In the phase space of (x, v) , the analytical solution traces out a perfect, closed ellipse, forever staying on this fixed physical "energy manifold".

Now, let's simulate this system using the explicit Euler method with a time step h :

$$\begin{aligned}x_{n+1} &= x_n + h \cdot v_n \\ v_{n+1} &= v_n + h \cdot \left(-\frac{k}{m}x_n\right)\end{aligned}$$

If we substitute this numerical update step back into our energy invariant function $I(x, v)$, we find:

$$I(x_{n+1}, v_{n+1}) = \frac{1}{2}m\left(v_n - h\frac{k}{m}x_n\right)^2 + \frac{1}{2}k(x_n + hv_n)^2$$

Expanding this expression reveals:

$$I(x_{n+1}, v_{n+1}) = I(x_n, v_n) + h^2\left(\frac{1}{2}\frac{k^2}{m}x_n^2 + \frac{1}{2}kv_n^2\right) \neq I(x_n, v_n)$$

The energy is not conserved. Because the error term on the right is strictly positive ($h^2>0, k>0, m>0$), the total calculated energy artificially increases at every single time step. Instead of tracing a stable, closed ellipse, the numerical solution spirals outward indefinitely. The solver has no built-in knowledge of the system's physics and lacks a mechanism to constrain its steps to the true energy manifold.



The **MassSpringOscillator** reference FMU implements this example and exposes $I(x, v)$ as an **<Invariant>**, see [reference-FMUs/MassSpringOscillator](#).

4.3.3. DAE Index Reduction and ODE with Invariants

The constraints are not always related to conservation laws. If the index of a DAE is reduced to zero to obtain an ODE, some equations may be omitted. In order to ensure that these equations are taken into account and used to avoid drift, these equations should be exploited.

The connection between an ODE solved with invariants and a DAE is fundamental, particularly when dealing with index reduction. Many physical systems, such as multibody dynamics, are naturally modeled as high-index DAEs. Reducing the DAE index to zero (transforming it into an ODE) simplifies the solution process but removes the original algebraic constraints from the solved equations. These constraints, which define the system's geometric invariants (e.g., the set of physically allowed positions), are then subject to numerical drift.

After sufficient differentiation, we often obtain a pure ODE $\dot{y} = F(y)$ for which the original algebraic constraint $g(y) = 0$ is no longer part of the differential equations. Instead, it becomes an invariant:

$$g(y(t)) = g(y(0))$$

Since the consistent initial condition satisfies $g(y(0)) = 0$, the reduced ODE should preserve $g(y(t)) = 0$. Thus, the algebraic constraints of the original DAE become invariants of the reduced ODE.

Consider the planar pendulum, an index-3 model constrained by the length of the rod. The full **index-3 DAE** in Cartesian coordinates is:

$$\begin{aligned}\dot{x} &= u, \\ \dot{y} &= v, \\ \dot{u} &= -\lambda x / m, \\ \dot{v} &= -\lambda y / m - g, \\ 0 &= x^2 + y^2 - L^2\end{aligned}$$

Index reduction eliminates λ and produces an ODE in (x, y, u, v) , but **removes the original constraints**. The resulting ODE no longer enforces:

- **Position invariant:** $g_1 = x^2 + y^2 - L^2$
- **Velocity invariant:** $g_2 = xu + yv$

If the invariants are ignored, numerical drift in the integration of the ODE alone causes, for example, the pendulum to **stretch or shrink** over time. To prevent numerical drift, **both constraints must be enforced** via projection after each integration step (or every few steps).

4.3.4. Invariant and Manifold Checking

This is where invariant and manifold checking (and correction) becomes essential. To preserve physical fidelity, simulations must actively monitor and correct for drift using one of two strategies.

4.3.4.1. State Augmentation (Continuous Correction)

In this approach the invariants are incorporated in the ODE using a Lagrange multiplier. If numerical errors cause $g(y) \neq 0$, the trajectory slowly leaves the constraint manifold. An invariant-preserving formulation augments the ODE with a correction term:

$$\dot{y} = F(y) + G(y)\lambda$$

where λ is chosen so that:

$$\frac{d}{dt}g(y) = 0$$

This equation determines λ and ensures that the solution remains on the constraint manifold.

4.3.4.2. Geometric Projection (Discrete Correction)

In this method, at regular intervals during the simulation, we calculate and monitor the value of the known invariants using the current numerical state. For example, we compute $I(x_n, v_n)$ and monitor its change from the initial value $I(x_0, v_0)$. A significant deviation is a direct measure of the simulation's accumulated error with respect to the physics of the problem.

For critical simulations, the **projection method** is a popular method where the correction is done during the integration. With standard numerical solvers (multi-step or single step), after a few steps or even after every step, the solution is "projected" back onto the correct invariant manifold. For instance, the state (x_n, v_n) is minimally adjusted to a new state $(\tilde{x}_n, \tilde{v}_n)$ such that $I(\tilde{x}_n, \tilde{v}_n) = I(x_0, v_0)$. Some off-the-shelf open-source numerical solvers support this feature.

For multiple invariants, the correction is obtained by solving a small constrained least-squares problem involving the Jacobian of all constraints, replacing the single invariant $I(x)$ with a vector of m constraints $H(x) \in \mathbb{R}^m$. For m constraints $H(x) \in \mathbb{R}^m$, the correction is:

$$\tilde{x}_n = x_n - G^T(GG^T)^{-1}H(x_n), \quad G = \frac{\partial H}{\partial x}(x_n) \in \mathbb{R}^{m \times n}$$

This is the **orthogonal projection** onto the intersection of all constraint manifolds. [\[Brenan\]](#), [\[Hairer1\]](#), [\[Hairer2\]](#)

References

- [cui2020mass] Cui, Hantao, Fangxing Li, and Joe H. Chow. "Mass-matrix differential-algebraic equation formulation for transient stability simulation." arXiv preprint arXiv:2008.03883, 2020.
- [cellier2006continuous] Cellier, François E., and Ernesto Kofman. "Continuous system simulation". Boston, MA: Springer US, 2006.
- [hairer2006numerical] Hairer, Ernst, Christian Lubich, and Michel Roche. "The numerical solution of differential-algebraic systems by Runge-Kutta methods". Vol. 1409. Springer, 2006.
- [cellier1993automated] Cellier, Francois E., and Hilding Elmqvist. "Automated formula manipulation supports object-oriented continuous-system modeling". IEEE Control Systems Magazine 13.2 (1993): 28-38.
- [braun2017solving] Willi Braun, Francesco Casella, and Bernhard Bachmann. "Solving Large-Scale Modelica Models: New Approaches and Experimental Results Using OpenModelica". *Linköping Electronic Conference Proceedings (Print)*, pp. 557-563. Linköping University Electronic Press, 2017. <https://re.public.polimi.it/bitstream/11311/1065357/1/2017-BraunCasellaBachmann.pdf>
- [casella2015simulation] Casella, Francesco. "Simulation of Large-Scale Models in Modelica: State of the Art and Future Perspectives". Proceedings of the 11th International Modelica Conference, pp. 459-468, 2015. <https://dx.doi.org/10.3384/ecp15118459>
- [PW13] Preston-Werner, T. (2013): **Semantic Versioning 2.0.0**. <https://semver.org/spec/v2.0.0.html>
- [Robertson1966] Robertson, H. H. "The Solution of a Set of Reaction Rate Equations". In: J. Walsh (ed.), *Numerical Analysis: An Introduction*, pp. 178–182. Academic Press, 1966.
- [Hindmarsh2026IDAExamples] Hindmarsh, Alan C., Radu Serban, and Aaron Collier. *Example Programs for IDA v7.8.0*. Technical Report UCRL-SM-208113. Lawrence Livermore National Laboratory, 2026.
- [Brenan] Brenan, K. E., Campbell, S. L., Petzold, L. R. (1996) "Numerical Solution of Initial-Value Problems in Differential-Algebraic Equations" SIAM.
- [Hairer1] Hairer, E., Wanner, G. (1996). "Solving Ordinary Differential Equations II: Stiff and Differential-Algebraic Problems" Springer.
- [Hairer2] Hairer, E., Lubich, C., & Wanner, G. "Geometric Numerical Integration: Structure-Preserving Algorithms for Ordinary Differential Equations" Springer.
- [MN2018] NAJAFi M. "Simulation of high-index DAEs and ODEs with constraints in FMI" Proceedings of the 2nd Japanese Modelica Conference Tokyo, Japan, 2018.

[1] Source: https://en.wikipedia.org/wiki/Differential-algebraic_system_of_equations

[2] e.g. by using Pantelides algorithm, see [cellier1993automated]

[3] Modelica Standard Library version 4.1.0

[4] <https://github.com/casella/ScalableTestSuite>, see also [casella2015simulation]

[5] <https://computing.llnl.gov/projects/sundials/ida>